

# Boundary-Continuous Cross-Camera RGB Mapping via Hue-Split Model Trees

Yuma Kinoshita\* and Hitoshi Kiya†

\* Tokai University, Japan

† Tokyo Metropolitan University, Japan

**Abstract**—We propose a hue-split model-tree method for boundary-continuous cross-camera RGB mapping. Cross-camera RGB mapping aims to produce consistent color representations across cameras whose recorded RGB values differ due to sensor spectral sensitivities and image-signal processing pipelines. A common chart-based remedy is to estimate a single global affine color correction matrix (CCM), but such a global model cannot capture hue-specific discrepancies between cameras. To capture that behavior, we recursively partition the source-camera color space along a scalar hue coordinate and build a model tree that stores an affine CCM at every node. For fitting the node CCMs, we utilize a log-domain error objective. To prevent false contours that arise from hard hue splits, we further introduce a boundary-continuous formulation in which the prediction is obtained by blending the log-domain outputs of all node CCMs along the root-to-leaf path. The path-wise blending weights are optimized under a simplex constraint using both a chart-pair fitting loss and an explicit continuity regularizer defined on deterministic boundary prototype pairs placed just on either side of each learned hue threshold. We conducted an experiment on a Canon EOS-1Ds Mark II to Canon EOS 20D mapping using the Middlebury Registered Color Checker dataset. The results show that hue splitting substantially reduces log-RMSE over a single global affine CCM and that the proposed path blending with boundary prototype regularization simultaneously improves accuracy and suppresses chromaticity gaps at the learned hue thresholds across two illuminants and multiple exposure conditions.

**Index Terms**—Color correction, cross-camera color consistency, hue segmentation, model trees, continuity regularization

## I. INTRODUCTION

Color is one of the most fundamental cues exploited in computer vision. Object recognition, image segmentation, visual tracking, and scene understanding all rely on the assumption that the color of a surface is a stable, reproducible quantity. In practice, however, the RGB values recorded for a given surface depend on sensor spectral sensitivities, the optics, and the subsequent image-signal processing pipeline [1]. Because these factors differ from device to device, two cameras photographing the same scene under the same illumination can produce noticeably different RGB triplets, which complicates multi-camera acquisition, content reuse across devices, and reproducible image analysis. This paper focuses on the cross-camera RGB mapping for obtaining consistent color representations across different cameras.

A practical remedy is chart-based calibration, where paired color measurements from a source camera and a target camera are used to estimate a direct RGB-to-RGB mapping. The mapping is often implemented as a linear or affine transform

by a single global color correction matrix (CCM). Such simple models remain attractive and widely used in calibration workflows [2] because they are easy to estimate and computationally light. However, a single global CCM cannot capture hue-specific discrepancies between cameras, so the correction error tends to be unevenly distributed across hues.

This paper addresses that limitation with a *hue-split model tree*. A model tree is a decision tree whose leaves hold local regression models instead of constant predictions [3], [4]. In our formulation, every node of the tree stores a  $3 \times 4$  affine CCM that maps source-camera RGB to target-camera RGB. We construct the tree by recursively splitting the source-camera RGB color space along a hue coordinate: at current leaf node, we search the hue threshold that minimizes the total post-split fitting loss for adding child nodes, and then estimate a dedicated affine CCM for each child node. Splitting continues until the maximum log-domain error of chart-color pairs corresponding to each leaf falls below a tolerance. Because different hue partitions are now served by their own affine maps, the tree captures hue-dependent inter-camera discrepancies that a single global CCM cannot model, while remaining interpretable and lightweight.

To reduce false contours due to hard hue splits, we also propose a boundary-continuous formulation with two key ideas. First, inference blends the log-domain estimates from all node CCMs on the selected root-to-leaf path, rather than using only the terminal leaf estimate. Second, the blending weights are optimized not only for prediction accuracy on chart pairs but also for continuity across every hue split. For that purpose, we introduce deterministic boundary prototype pairs, i.e., synthetic source-camera colors placed just to the left and right of a split. Penalizing their output discrepancy yields a direct, split-aware continuity regularizer.

Our contributions are threefold:

- We formulate cross-camera RGB mapping as a deterministic hue-split model tree with node-wise affine regressors in the log domain.
- We introduce deterministic boundary prototype pairs and a path-weight optimization scheme that explicitly penalizes discontinuities at learned hue thresholds.
- We validate the proposed method on the Middlebury Registered Color Checker dataset, and it is shown that hue splitting with boundary prototype regularization improves cross-camera mapping accuracy and boundary continuity under multiple illuminants and exposure conditions.

## II. RELATED WORK

### A. Global Chart-Based Camera Mapping

Classical camera characterization often maps camera RGB values to a device-independent color space or another device using a global regression model estimated from chart measurements. Constrained least-squares regression and related  $3 \times 3$  transforms remain a standard baseline because of their simplicity and robust behavior under calibration constraints [2]. Higher-order polynomial mappings can reduce fitting error by expanding the feature set beyond the raw RGB triplet [5]. Comparative studies have also shown that neural networks and polynomial transforms can offer similar accuracy, with polynomial models often preferred for ease of training and deployment [6].

More recently, root-polynomial regression has been proposed as a nonlinear extension that retains desirable scaling behavior better than ordinary polynomial expansion [7]. These methods are effective when one smooth global transform is adequate, but they still optimize a single set of coefficients for all colors. As a result, they do not explicitly model region-specific behavior that may arise when inter-camera discrepancies vary with hue.

### B. Hue-Aware and Localized Color Correction

Several studies have observed that partitioning the color domain can improve approximation accuracy. Find Andersen and Hardeberg proposed a hue-plane-preserving approach that uses multiple  $3 \times 3$  matrices for different subsets of camera responses [8]. More recently, Li *et al.* showed that grouping training samples by hue angle can improve conversion accuracy for wide-color-gamut cameras [9]. These results motivate local modeling: if different hue regions exhibit different mapping behavior, a set of local regressors can be more accurate than a single global matrix.

However, local color correction also introduces a practical problem. When neighboring regions are calibrated independently, their predictions need not agree near the partition boundary. For image-wide application, even small output jumps can produce visible false contours on smooth gradients. Therefore, a localized mapping should ideally improve fidelity without sacrificing continuity.

### C. Model Trees and Smooth Tree-Based Prediction

Model trees address nonlinear regression by recursively partitioning the input space and assigning a simple regressor to each region. Quinlan’s M5 framework and its later formulation by Wang and Witten established model trees as an interpretable alternative to purely global regression [3], [4]. Because each leaf contains a local parametric model, model trees are well suited to problems that are globally nonlinear but locally simple.

A related line of work seeks smoother behavior by softly combining local experts. Hierarchical mixtures of experts replace hard decisions with probabilistic gating and weighted expert outputs [10]. Such models provide smooth transitions,

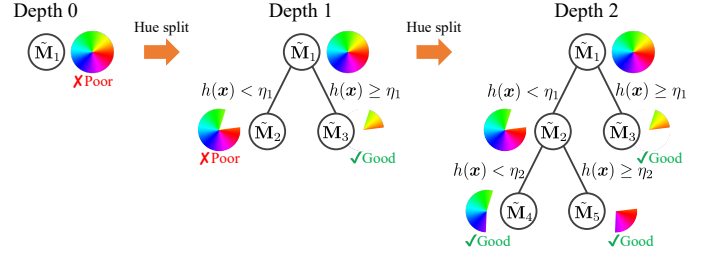


Fig. 1: Building hue-split model tree

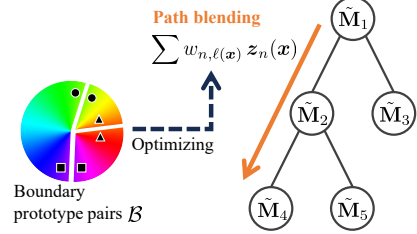


Fig. 2: Path blending with boundary prototype regularization

but they generally require learning both gating and expert functions jointly and are not tailored to deterministic hue partitions derived from chart calibration.

Our approach combines the interpretability of a deterministic hue-split tree with a lightweight smoothing mechanism. Instead of replacing the tree with a fully probabilistic gating network, we keep the explicit hue intervals and optimize path-wise blending weights after the local affine models have been estimated. Moreover, continuity is enforced by boundary prototype pairs that directly target the split locations responsible for visible discontinuities.

## III. PROPOSED METHOD

Given  $K$  paired chart measurements from a source camera and a target camera,

$$\mathcal{D} = \{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^K, \quad \mathbf{x}_i, \mathbf{y}_i \in \mathbb{R}^3, \quad (1)$$

our goal is to estimate a mapping  $f: \mathbb{R}_+^3 \rightarrow \mathbb{R}_+^3$  that satisfies two requirements simultaneously:

- 1) **Accuracy.**  $f(\mathbf{x}_i)$  reproduces the target-camera RGB  $\mathbf{y}_i$  for every chart pair.
- 2) **Boundary continuity.**  $f$  varies smoothly across the entire source-camera color domain.

To pursue both requirements, we model  $f$  as a piecewise affine transform organized by a hue-split model tree. We augment each input with a bias term,  $\tilde{\mathbf{x}}_i = [\mathbf{x}_i^\top 1]^\top \in \mathbb{R}^4$ , and assign an affine matrix  $\tilde{\mathbf{M}} \in \mathbb{R}^{3 \times 4}$  to every tree node so that a node predicts  $\tilde{\mathbf{M}}\tilde{\mathbf{x}}$ . We also define a scalar hue coordinate  $h(\mathbf{x})$  extracted from the source-camera RGB after white balancing and HSV conversion;  $h$  serves as the sole splitting variable of the tree.

### A. Hue Coordinate

The tree partitions the source-camera color space by a scalar hue coordinate  $h(\mathbf{x})$ . We obtain  $h$  from the source RGB  $\mathbf{x}$  via

white balancing followed by conversion to HSV color space. White balancing is simply done using the source RGB  $\mathbf{x}_{\text{white}}$  of a white patch in the chart as

$$\mathbf{x}^{\text{wb}} = \mathbf{x} \oslash \mathbf{x}_{\text{white}}, \quad (2)$$

where  $\oslash$  denotes element-wise division.

Writing  $\mathbf{x}^{\text{wb}} = (x_R, x_G, x_B)$  and defining

$$C_{\max} = \max(x_R, x_G, x_B), \quad (3)$$

$$C_{\min} = \min(x_R, x_G, x_B), \quad (4)$$

$$\Delta = C_{\max} - C_{\min}, \quad (5)$$

conversion to the value  $V$ , saturation  $S$ , and hue  $H$  are defined as

$$V = C_{\max}, \quad (6)$$

$$S = \begin{cases} \Delta/C_{\max} & \text{if } C_{\max} > 0, \\ 0 & \text{otherwise,} \end{cases} \quad (7)$$

$$H = \begin{cases} 0^\circ & \text{if } \Delta = 0, \\ 60^\circ \times \frac{x_G - x_B}{\Delta} \bmod 360^\circ & \text{if } C_{\max} = x_R, \\ 60^\circ \times \frac{x_B - x_R}{\Delta} + 120^\circ & \text{if } C_{\max} = x_G, \\ 60^\circ \times \frac{x_R - x_G}{\Delta} + 240^\circ & \text{if } C_{\max} = x_B. \end{cases} \quad (8)$$

We define the hue coordinate as

$$h(\mathbf{x}) = \begin{cases} H & \text{if } S \geq s_{\min}, \\ 0 & \text{otherwise.} \end{cases} \quad (9)$$

The hue coordinate  $h(\mathbf{x})$  is used as a device-dependent feature for branch selection and is not intended to represent a colorimetrically accurate estimate of perceptual hue. Although  $S$  and  $V$  are not used for tree splitting, they are used to construct the boundary prototype pairs in Section III-D.

### B. Hue-Split Model-Tree Construction

For a node  $n$  with chart-color index set  $\mathcal{S}_n \subset \{1, \dots, K\}$ , we first fit its affine CCM  $\tilde{\mathbf{M}}_n$  by following the log-domain error minimization:

$$\tilde{\mathbf{M}}_n = \tilde{\mathbf{M}}^*(\mathcal{S}_n) = \arg \min_{\tilde{\mathbf{M}}} \mathcal{L}(\tilde{\mathbf{M}}; \mathcal{S}), \quad (10)$$

$$\mathcal{L}(\tilde{\mathbf{M}}; \mathcal{S}) = \frac{1}{3|\mathcal{S}|} \sum_{i \in \mathcal{S}} \left\| \log_2([\tilde{\mathbf{M}}\tilde{\mathbf{x}}_i]_+ + \epsilon) - \log_2(\mathbf{y}_i + \epsilon) \right\|_2^2, \quad (11)$$

where  $[\cdot]_+$  denotes the element-wise positive-part operator (i.e.,  $[z]_+ = \max(z, 0)$  for a scalar  $z$ ) and  $\epsilon > 0$  is a small constant to avoid zero arguments to the logarithm. In our implementation, the affine CCM is optimized with a Gauss-Newton method initialized from the least-squares solution in the linear domain.

We then evaluate the node's maximum training error,

$$\varepsilon_{\max}(n) = \max_{i \in \mathcal{S}_n} \frac{1}{3} \left\| \log_2([\tilde{\mathbf{M}}_n\tilde{\mathbf{x}}_i]_+ + \epsilon) - \log_2(\mathbf{y}_i + \epsilon) \right\|_2^2. \quad (12)$$

If  $\varepsilon_{\max}(n) \leq \tau$ , or if the node violates practical stopping conditions such as a minimum leaf size or a maximum depth, the node becomes a leaf.

Otherwise, the node is split by a hue threshold  $\eta$ . Let

$$\mathcal{S}_L(\eta) = \{i \in \mathcal{S}_n \mid h(\mathbf{x}_i) < \eta\}, \quad (13)$$

$$\mathcal{S}_R(\eta) = \{i \in \mathcal{S}_n \mid h(\mathbf{x}_i) \geq \eta\}, \quad (14)$$

under an assumption that  $\{\mathbf{x}_i\}$  are sorted by hue. Among admissible candidate thresholds  $\eta \in \mathcal{T}_n$ , formed by the midpoints of sorted hue values, we choose the one that minimizes the total post-split loss,

$$\eta_n = \arg \min_{\eta \in \mathcal{T}_n} \left( |\mathcal{S}_L(\eta)| \mathcal{L}(\tilde{\mathbf{M}}^*(\mathcal{S}_L(\eta)); \mathcal{S}_L(\eta)) + |\mathcal{S}_R(\eta)| \mathcal{L}(\tilde{\mathbf{M}}^*(\mathcal{S}_R(\eta)); \mathcal{S}_R(\eta)) \right). \quad (15)$$

The above procedure is started at the root node with  $\mathcal{S}_1 = \{1, \dots, K\}$  and it is then applied recursively to the left and right children. This produces a deterministic hue-split model tree in which each node stores its own affine CCM.

Compared with a single CCM, the tree can approximate different local mappings in different hue intervals. Compared with arbitrary nonlinear regressors, the resulting representation remains interpretable: the tree explicitly reveals where the source-camera hue domain has been partitioned and which local affine map is responsible for each region.

### C. Path Blending in the Log Domain

Using only the leaf CCM can make the output discontinuous at a split boundary because adjacent leaves are fitted almost independently. We therefore blend all estimates on the root-to-leaf path. Let  $\ell(\mathbf{x})$  be the leaf selected by the hue-based routing rule for input  $\mathbf{x}$ , and let  $\mathcal{P}_{\ell(\mathbf{x})}$  denote the set of nodes on the corresponding path.

For each node  $n$  on that path, we define the local log-domain prediction

$$\mathbf{z}_n(\mathbf{x}) = \log_2([\tilde{\mathbf{M}}_n\tilde{\mathbf{x}}]_+ + \epsilon), \quad (16)$$

and we blend those predictions with path-specific nonnegative weights  $w_{n,\ell(\mathbf{x})} \geq 0$  as the final log-domain estimate  $\hat{\mathbf{z}}(\mathbf{x})$ ,

$$\hat{\mathbf{z}}(\mathbf{x}) = \sum_{n \in \mathcal{P}_{\ell(\mathbf{x})}} w_{n,\ell(\mathbf{x})} \mathbf{z}_n(\mathbf{x}) \quad \text{s.t.} \quad \sum_{n \in \mathcal{P}_{\ell(\mathbf{x})}} w_{n,\ell(\mathbf{x})} = 1. \quad (17)$$

The RGB prediction in the target-camera domain is then obtained by inverse log mapping,

$$\bar{\mathbf{y}}(\mathbf{x}) = 2^{\hat{\mathbf{z}}(\mathbf{x})} - \epsilon, \quad (18)$$

followed by clipping to the valid target RGB range.

### D. Blending Weight Optimization with Continuity Regularization

To optimize the weights  $\{w_n\}$ , we use not only the chart-fitting error but also an explicit boundary continuity term. Consider an internal node with hue threshold  $\eta$ . For that threshold, we construct a deterministic set of boundary prototype pairs,

$$\mathcal{B}_\eta = \{(\mathbf{b}_{\eta,m}^-, \mathbf{b}_{\eta,m}^+)\}_{m=1}^{M_\eta}, \quad (19)$$

where each pair is generated from a source color on the split boundary  $\eta$  by a small hue perturbation  $\delta$  in either direction: one is perturbed to  $\eta - \delta$  and the other to  $\eta + \delta$ , with saturation and value held fixed. Thus, the pair straddles the split boundary while remaining otherwise comparable.

The objective for weight optimization consists of three terms: a fitting term that encourages accurate chart prediction, a regularizer that penalizes discontinuity at the hue boundaries, and a small  $\ell_2$  stabilization term. The total objective becomes

$$J(\mathbf{w}) = \frac{1}{2}\mathcal{E}(\mathbf{w}) + \frac{\lambda}{2}\mathcal{R}(\mathbf{w}) + \frac{\xi}{2}\|\mathbf{w}\|_2^2, \quad (20)$$

where  $\lambda > 0$  controls the trade-off between prediction accuracy and continuity, and  $\xi > 0$  is a small stabilization parameter. The fitting term  $\mathcal{E}$  is given by the average log-domain MSE over the chart pairs as

$$\mathcal{E}(\mathbf{w}) = \frac{1}{3K} \sum_{i=1}^K \|\bar{\mathbf{z}}(\mathbf{x}_i) - \log_2(\mathbf{y}_i + \epsilon)\|_2^2, \quad (21)$$

while the boundary continuity term  $\mathcal{R}$  is given by the average squared log-domain jump over the boundary prototype pairs as

$$\mathcal{R}(\mathbf{w}) = \frac{1}{3|\mathcal{B}|} \sum_{(\mathbf{b}^-, \mathbf{b}^+) \in \mathcal{B}} \|\bar{\mathbf{z}}(\mathbf{b}^-) - \bar{\mathbf{z}}(\mathbf{b}^+)\|_2^2, \quad (22)$$

where  $\mathcal{B} = \bigcup_{\eta} \mathcal{B}_{\eta}$  is the union of all boundary prototype pairs across all splits.

For a fixed tree and fixed node CCMs,  $J(\mathbf{w})$  is convex in the weights because both (21) and (22) are quadratic in  $\mathbf{w}$  and the feasible set induced by the simplex constraint in Eq. (17) is convex. Therefore, we optimize  $\mathbf{w}$  by projected gradient descent as

$$\mathbf{w}^{(t+1)} = \Pi_{\mathcal{C}}(\mathbf{w}^{(t)} - \alpha \nabla J(\mathbf{w}^{(t)})), \quad (23)$$

where  $\Pi_{\mathcal{C}}$  is the projection onto the set  $\mathcal{C} = \{\mathbf{w} \mid w_n \geq 0, \sum_{n \in \mathcal{P}_t} w_n = 1, \forall \ell\}$  and  $\alpha > 0$  is a step size.

#### IV. EXPERIMENT

We conducted an experiment using real data to answer the following questions: 1) whether hue-wise local modeling improves cross-camera RGB mapping accuracy over a single global affine CCM, and 2) whether the proposed path blending with boundary prototype regularization reduces discontinuities at learned hue thresholds.

##### A. Experimental Conditions

We evaluated accuracy and boundary continuity of color mapping from Canon EOS-1Ds Mark II to Canon EOS 20D, using a raw-image pair from the Middlebury Registered Color Checker dataset [11]. In the dataset, we used the checker140s-RAW-JPG archive, kept the RAW-derived linear PNGs with wb1, extracted the inner 96 Digital ColorChecker SG patches. We built our model tree using RGB values of the 96 patches at the exposure condition of 0 EV while testing RGB values at  $v \in \{-1, 0, 1\}$  EV under both available light conditions: i1 and i2.

We compared the following three mapping methods:

- 1) Single global affine CCM (NoSplit-NoBlend): using  $\tilde{\mathbf{M}}_1$  fitted at the root node for all input colors without hue splitting.
- 2) Hard hue-split model tree without path blending (HueSplit-NoBlend): using only the leaf CCM for prediction without any path blending or continuity regularization.
- 3) Hue-split model tree with fixed M5-style path blending (HueSplit-M5Blend): using the M5 heuristic [3] for path blending weights without any continuity regularization.
- 4) Proposed hue-split model tree with path blending (HueSplit-OptimizedBlend): using the path blending and boundary prototype regularization with  $\lambda \in \{0, 0.1, 1\}$ .

All tree-based methods shared the same tree structure including hue threshold and node CCMs. During building the tree, we set the maximum depth to 2, the minimum leaf size to 4, and the max-error tolerance  $\tau$  to 0 (i.e., no early stopping based on error).

For learned path blending, we use  $\epsilon = 10^{-6}$ ,  $\xi = 10^{-6}$ , and a set  $\mathcal{B}$  of deterministic boundary prototype pairs generated with hue perturbation  $\delta = 1^\circ$  on a  $5 \times 4$  saturation-value grid, where  $S \in [0.05, 1.0]$  and  $V \in [0.04, 0.16]$ . The achromatic threshold is set to  $s_{\min} = 10^{-8}$ .

Accuracy is measured by the patch-wise log-root-mean-squared error (log-RMSE) as

$$d(\hat{\mathbf{y}}, \mathbf{y}) = \sqrt{\frac{1}{3}} \|\log_2(\hat{\mathbf{y}} + \epsilon) - \log_2(\mathbf{y} + \epsilon)\|_2, \quad (24)$$

and boundary smoothness is evaluated by the average boundary jump

$$B = \frac{1}{|\mathcal{B}|} \sum_{(\mathbf{b}^-, \mathbf{b}^+) \in \mathcal{B}} \sqrt{\frac{1}{3}} \|\hat{\mathbf{z}}(\mathbf{b}^-) - \hat{\mathbf{z}}(\mathbf{b}^+)\|_2, \quad (25)$$

where  $\mathcal{B}$  is the same set of deterministic boundary prototype pairs used for regularization in Eq. (22).

##### B. Results

Figures 3 and 4 visualize the corresponding 0 EV correction under i1 and the +2 EV chromaticity gap under the same light at depth 2, respectively. From Fig. 4, we can see that HueSplit-NoBlend created a visible chromaticity gap at the learned hue threshold, and M5 and optimized blends with  $\lambda = 0.1$  and  $\lambda = 1.0$  reduced the gap.

Table I summarizes the accuracy and boundary continuity under light condition i1. At the training exposure of 0 EV, hue splitting yields a substantial reduction in log-RMSE over the single global affine CCM: the error drops from 0.8426 for NoSplit-NoBlend to 0.4977 at depth 1 and further to 0.2110 at depth 2 for HueSplit-NoBlend. This improvement shows that hue-wise local modeling captures non-uniform color shifts that a single global affine map cannot represent. Comparing the three variants of HueSplit-OptimizedBlend with HueSplit-NoBlend under the same depth



Fig. 3: Qualitative comparison of corrected chart patches for the Middlebury Color Checker dataset under light  $i_1$  at 0 EV and depth 2.

condition, the proposed HueSplit-OptimizedBlend consistently improves both accuracy and continuity while HueSplit-M5Blend sometimes degrades accuracy relative to HueSplit-NoBlend. For this results, we can see that the boundary prototype regularization effectively suppresses discontinuities at the learned hue thresholds without sacrificing accuracy.

Table II reports the same comparison under light condition  $i_2$ , and overall the same trends are observed. At the training exposure of 0 EV, hue splitting again yields a substantial reduction in log-RMSE over the single global affine CCM: the error drops from 1.5016 for NoSplit-NoBlend to 1.3138 at depth 1 and further to 1.1565 at depth 2 for HueSplit-NoBlend. Comparing the three variants of HueSplit-OptimizedBlend with HueSplit-NoBlend under the same depth condition, the proposed method consistently improves both accuracy and continuity at depth 2 across all exposures, whereas HueSplit-M5Blend provides at best a marginal change relative to HueSplit-NoBlend. A notable difference from light  $i_1$  appears at depth -1 EV, where HueSplit-NoBlend (1.7858) is in fact worse than the single global CCM (1.5766); the proposed method with  $\lambda = 1.0$  recovers this loss to 1.6142, and at depth 2 it achieves the best log-RMSE of 1.5553, outperforming the global baseline. These results confirm that

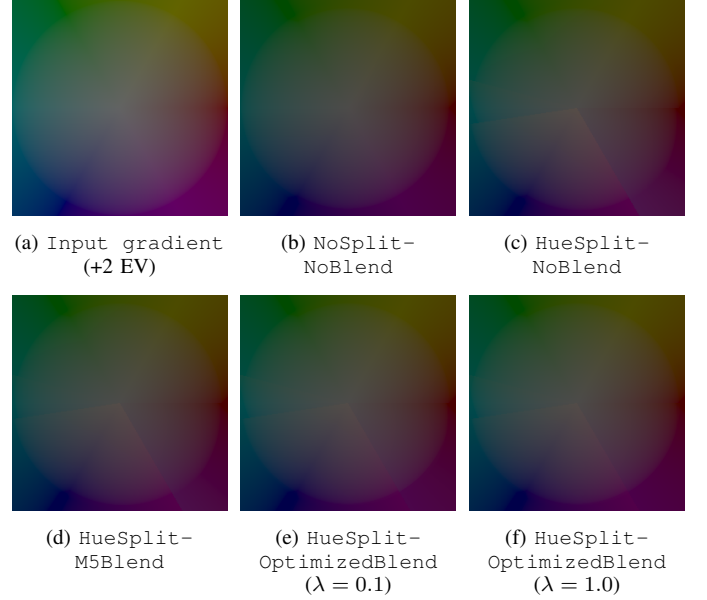


Fig. 4: Chromaticity-gap visualization for methods trained on the Middlebury Color Checker dataset under light  $i_1$ , by applying the methods to a +2 EV gradient image. The depth is 2 for all tree-based methods.

the boundary prototype regularization remains effective under a different illumination.

## V. CONCLUSION

We presented a boundary-continuous hue-split model tree for cross-camera RGB mapping. The tree recursively partitions the source-camera color space along a hue coordinate and stores a node-wise  $3 \times 4$  affine CCM fitted in the log domain, while a path-blending scheme with deterministic boundary prototype pairs explicitly penalizes log-domain jumps at learned hue thresholds. Experiments on a Canon EOS-1Ds Mark II to Canon EOS 20D mapping using the Middlebury Registered Color Checker dataset showed that hue splitting substantially reduces log-RMSE over a single global affine CCM, and that the proposed boundary prototype regularization consistently improves both accuracy and continuity at depth 2 across all tested exposures and both illuminants, whereas the M5-style fixed blending provided only marginal continuity gains.

Limitations of this work include the evaluation on a single camera pair with 96 chart patches under two illuminants. Future work will validate the method on a wider range of camera pairs and illuminants, compare with broader nonlinear baselines, and assess perceptual quality on full-resolution natural images. We will also examine whether incorporating sensor spectral sensitivities and calibrated color-balance transforms improves illuminant robustness and hue-split stability.

## REFERENCES

- [1] R. Ramanath, W. E. Snyder, Y. Yoo, and M. S. Drew, “Color image processing pipeline,” *IEEE Signal Pro-*

TABLE I: Log-RMSE and boundary jump  $B$  of cross-camera RGB mapping from Canon EOS-1Ds Mark II to EOS 20D on the Middlebury Color Checker dataset under illuminant  $\mathbf{i}_1$ . Lower is better for both metrics. Models are trained at 0 EV. "Depth" indicates the test-time depth at which the root-to-leaf path through the model tree is truncated.

| Method                                      | Depth | −1 EV         |               | 0 EV          |               | +1 EV         |               |
|---------------------------------------------|-------|---------------|---------------|---------------|---------------|---------------|---------------|
|                                             |       | log-RMSE      | $B$ [EV]      | log-RMSE      | $B$ [EV]      | log-RMSE      | $B$ [EV]      |
| NoSplit-NoBlend                             | –     | 2.6888        | –             | 0.8426        | –             | <b>1.2563</b> | –             |
| HueSplit-NoBlend                            | 1     | 2.5118        | 0.1555        | 0.4977        | 0.0766        | 1.2751        | 0.0590        |
| HueSplit-M5Blend [3]                        | 1     | 2.5096        | <b>0.1508</b> | 0.6389        | <b>0.0714</b> | 1.2728        | 0.0544        |
| HueSplit-OptimizedBlend ( $\lambda = 0$ )   | 1     | 2.5060        | 0.1814        | 0.4852        | 0.0755        | 1.2674        | <b>0.0543</b> |
| HueSplit-OptimizedBlend ( $\lambda = 0.1$ ) | 1     | 2.5060        | 0.1814        | 0.4852        | 0.0755        | 1.2674        | <b>0.0543</b> |
| HueSplit-OptimizedBlend ( $\lambda = 1.0$ ) | 1     | 2.5060        | 0.1814        | 0.4852        | 0.0755        | 1.2674        | <b>0.0543</b> |
| HueSplit-NoBlend                            | 2     | 3.2930        | 2.3387        | 0.2110        | 0.7737        | 1.3893        | 0.8382        |
| HueSplit-M5Blend [3]                        | 2     | 3.2416        | 1.9803        | 0.3588        | 0.5482        | 1.3693        | 0.4659        |
| HueSplit-OptimizedBlend ( $\lambda = 0$ )   | 2     | 2.6210        | 2.2097        | <b>0.1653</b> | 0.6813        | 1.3224        | 0.7451        |
| HueSplit-OptimizedBlend ( $\lambda = 0.1$ ) | 2     | 2.4046        | 1.8609        | 0.1687        | 0.5715        | 1.3192        | 0.6440        |
| HueSplit-OptimizedBlend ( $\lambda = 1.0$ ) | 2     | <b>2.2118</b> | 1.1992        | 0.1810        | 0.3753        | 1.3132        | 0.4546        |

TABLE II: Log-RMSE and boundary jump  $B$  of cross-camera RGB mapping from Canon EOS-1Ds Mark II to EOS 20D on the Middlebury Color Checker dataset under illuminant  $\mathbf{i}_2$ . Lower is better for both metrics. Models are trained at 0 EV. "Depth" indicates the test-time depth at which the root-to-leaf path through the model tree is truncated.

| Method                                      | Depth | −1 EV         |               | 0 EV          |               | +1 EV         |               |
|---------------------------------------------|-------|---------------|---------------|---------------|---------------|---------------|---------------|
|                                             |       | log-RMSE      | $B$ [EV]      | log-RMSE      | $B$ [EV]      | log-RMSE      | $B$ [EV]      |
| NoSplit-NoBlend                             | –     | 1.5766        | –             | 1.5016        | –             | 2.2360        | –             |
| HueSplit-NoBlend                            | 1     | 1.7858        | 0.8853        | 1.3138        | 0.4597        | 2.2355        | 0.2395        |
| HueSplit-M5Blend [3]                        | 1     | 1.6274        | <b>0.4100</b> | 1.3197        | 0.1470        | 2.2354        | 0.0833        |
| HueSplit-OptimizedBlend ( $\lambda = 0$ )   | 1     | 1.7858        | 0.8853        | 1.3138        | 0.4597        | 2.2355        | 0.2395        |
| HueSplit-OptimizedBlend ( $\lambda = 0.1$ ) | 1     | 1.6359        | 0.6327        | 1.3152        | 0.2573        | 2.2357        | 0.1406        |
| HueSplit-OptimizedBlend ( $\lambda = 1.0$ ) | 1     | 1.6142        | 0.5439        | 1.3165        | <b>0.1265</b> | 2.2362        | <b>0.0741</b> |
| HueSplit-NoBlend                            | 2     | 1.7731        | 1.7012        | 1.1565        | 0.9983        | 2.2095        | 0.5067        |
| HueSplit-M5Blend [3]                        | 2     | 1.7677        | 1.5259        | 1.1555        | 0.8675        | 2.2072        | 0.4253        |
| HueSplit-OptimizedBlend ( $\lambda = 0$ )   | 2     | 1.7484        | 1.3703        | <b>1.1390</b> | 0.8061        | 2.2082        | 0.4114        |
| HueSplit-OptimizedBlend ( $\lambda = 0.1$ ) | 2     | 1.5978        | 0.9998        | 1.1452        | 0.4790        | <b>2.1993</b> | 0.2677        |
| HueSplit-OptimizedBlend ( $\lambda = 1.0$ ) | 2     | <b>1.5553</b> | 0.9077        | 1.1496        | 0.3668        | 2.1998        | 0.2197        |

- cessing Magazine, vol. 22, no. 1, pp. 34–43, 2005. DOI: 10.1109/MSP.2005.1407713.
- [2] G. D. Finlayson and M. S. Drew, "Constrained least-squares regression in color spaces," *Journal of Electronic Imaging*, vol. 6, no. 4, pp. 484–493, 1997. DOI: 10.1117/12.278080.
- [3] R. J. Quinlan, "Learning with continuous classes," in *Proceedings of the 5th Australian Joint Conference on Artificial Intelligence*, Singapore: World Scientific, 1992, pp. 343–348.
- [4] Y. Wang and I. H. Witten, "Induction of model trees for predicting continuous classes," in *Poster Papers of the 9th European Conference on Machine Learning*, Springer, 1997.
- [5] G. Hong, M. R. Luo, and P. A. Rhodes, "A study of digital camera colorimetric characterization based on polynomial modeling," *Color Research & Application*, vol. 26, no. 1, pp. 76–84, 2001.
- [6] T. L. V. Cheung, S. Westland, D. R. Connah, and C. Ripamonti, "A comparative study of the characterisation of colour cameras by means of neural networks and polynomial transforms," *Coloration Technology*, vol. 120, pp. 19–25, 2004. DOI: 10.1111/j.1478-4408.2004.tb00201.x.
- [7] G. D. Finlayson, M. Mackiewicz, and A. Hurlbert, "Color correction using root-polynomial regression," *IEEE Transactions on Image Processing*, vol. 24, no. 5, pp. 1460–1470, 2015. DOI: 10.1109/TIP.2015.2405336.
- [8] C. F. Andersen and J. Y. Hardeberg, "Colorimetric characterization of digital cameras preserving hue planes," in *Proceedings of the IS&T/SID 13th Color Imaging Conference*, 2005, pp. 141–146. DOI: 10.2352/CIC.2005.13.1.art00028.
- [9] Y. Li, N. Liao, Y. Li, H. Li, and W. Wu, "Color conversion of wide-color-gamut cameras using optimal training groups," *Sensors*, vol. 23, no. 16, p. 7186, 2023. DOI: 10.3390/s23167186.
- [10] M. I. Jordan and R. A. Jacobs, "Hierarchical mixtures of experts and the EM algorithm," *Neural Computation*, vol. 6, no. 2, pp. 181–214, 1994. DOI: 10.1162/neco.1994.6.2.181.
- [11] Middlebury Computer Vision, *Middlebury registered color checker dataset*, <https://vision.middlebury.edu/color/data/>, Dataset webpage, accessed April 4, 2026, 2011.